

ICT600 CORRECTED STUDENT EDITION

February 2026 Corrected Student Scheme

Question-verified edition; no official answer scheme was supplied

Scope	February 2026 final examination
Evidence	Printed question, supplied scheme where available, current lecture notes, and technical verification
Status	100 marks: Part A 70, Part B 30

Prepared 26 September 2026

How to use this reference

Teaching-use document

Attempt the original question before revealing this scheme. This corrected edition is not an official UiTM replacement; it records known source errors and accepts equivalent technically correct answers.

- Assessment total: 100 marks: Part A 70, Part B 30.
- Model answers show one defensible solution; alternative correct wording or code may receive full credit.
- For code, method marks remain available when the approach is sound and a minor syntax error does not invalidate the solution.
- Security improvements are shown where historical examples trusted or displayed unvalidated input.

Q1

Four HTML functions and five web-based CMS tools.

[18 marks]

Model answer

- HTML functions: structure content, link/embed resources, build forms, and organise tabular/list content (2 each).
- Any five accepted course CMS/site-building tools with accurate identification (2 each).

Marking guide: Q1(a) 8; Q1(b) 10.

Q2

Website anatomy and client-side development tools.

[12 marks]

Model answer

- Containing block, logo, navigation, content, footer, whitespace (1 each).
- Any six valid development tools or clear category-example pairs (1 each).

Marking guide: 6 marks per subpart.

Q3(a)

Create a MySQL database using PHP.

[8 marks]

Model answer

- Connect (2), handle connection error (1), CREATE DATABASE (1), execute (1), success/database error handling (2), close (1).

Marking guide: Follow the full connection-execution-close sequence.

Q4

CodeIgniter Employee controller and MVC flow.

[10 marks]

Model answer

- CI3-compatible public index() loading employee_profile (4).
- Browser -> Employee controller/index -> load view -> Employee Profile -> rendered response to browser (6).

Marking guide: The supplied CI_Controller syntax indicates CodeIgniter 3.

Q5

Clear-text transmission and file-upload risks.

[12 marks]

Model answer

- Clear text: interception/tampering risk plus HTTPS/TLS and correctly configured certificates/no HTTP fallback (6).
- Upload: malicious file risk plus content/type validation, non-executable storage, and malware scanning (6).

Marking guide: Each subpart: risk 2 + two countermeasures 2 each.

Q3(b) - secure add.php exemplar

Study the complete working exemplar below.

[See printed allocation]

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
<?php
$conn = mysqli_connect('localhost', 'app_user', 'app_password', 'myDB');
if (!$conn) die('Connection failed: ' . mysqli_connect_error());

$username = trim($_POST['username'] ?? '');
$password = $_POST['password'] ?? '';
$role = $_POST['role'] ?? 'user';
$status = $_POST['status'] ?? 'active';
$allowedRoles = ['user', 'staff'];
$allowedStatuses = ['active', 'inactive'];

if ($username === '' || $password === '' ||
    !in_array($role, $allowedRoles, true) ||
    !in_array($status, $allowedStatuses, true)) {
    die('Invalid input.');
```

```
}

$passwordHash = password_hash($password, PASSWORD_DEFAULT);
$stmt = mysqli_prepare($conn,
    'INSERT INTO users (username, password, role, status) VALUES (?, ?, ?, ?)');
mysqli_stmt_bind_param($stmt, 'ssss', $username, $passwordHash, $role, $status);
echo mysqli_stmt_execute($stmt) ? 'New record created successfully' : 'Insert failed';
mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The generated answer concatenates raw form values into SQL, stores the password in clear text, and accepts arbitrary role/status values. This corrected exemplar uses validation, allowlists, password hashing, and a prepared statement.

Part B Q1 - secure rental calculation core

Study the complete working exemplar below.

[See printed allocation]

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
<?php
function e(string $v): string { return htmlspecialchars($v, ENT_QUOTES, 'UTF-8'); }
$name = trim($_POST['customerName'] ?? '');
$phone = trim($_POST['phone'] ?? '');
$package = $_POST['package'] ?? '';
$lateHours = filter_input(INPUT_POST, 'lateHours', FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 0]]);
$rates = ['B' => 15, 'A' => 23, 'P' => 100];
if ($name === '' || $phone === '' || !isset($rates[$package]) || $lateHours === false) {
    die('Invalid input.');
```

```
}
$lateCharge = $package === 'P' ? 0 : $lateHours * 10;
$total = $rates[$package] + $lateCharge;
?>

<h1>Customer Details</h1>
<p>Name: <?= e($name) ?></p>
<p>Phone: <?= e($phone) ?></p>
```

```
<p>Package: <?= e($package) ?></p>
<p>Late return: <?= $lateHours ?> hour(s)</p>
<p>Charges: RM<?= number_format($total, 2) ?></p>
```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The generated calculation is plausible, but it trusts missing/invalid POST values and echoes raw input. This version validates and encodes output.

Part B Q2 - cumulative number analyser core

Study the complete working exemplar below.

[\[See printed allocation\]](#)

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
const numbers = [];
function sendNumber() {
  const input = document.getElementById('number');
  const value = Number(input.value);
  if (input.value.trim() === '' || !Number.isFinite(value)) return;
  numbers.push(value);
  document.getElementById('values').textContent = numbers.join(', ');
  document.getElementById('maximum').textContent = Math.max(...numbers);
  document.getElementById('total').textContent =
    numbers.reduce((sum, number) => sum + number, 0);
  input.value = '';
  input.focus();
}
```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The generated logic is correct. textContent is used here because the outputs are text, not HTML.

Consistency rules

- Award equivalent technically correct syntax and wording.
- Do not reproduce a known historical error merely because it appears in a supplied scheme.
- Keep printed subpart totals fixed unless a documented moderation decision says otherwise.
- For scenario questions, connect the explanation or mitigation to the scenario.
- Record additional accepted alternatives for the next controlled revision.